Implementierung

Aufbau, Entscheidungen und Tests

Inhaltsverzeichnis

Aufbau 1
Dateien 2
Wichtige Codeteile 3
Neuron: Forward und Update 3
Layer: Backpropagation 3
Netz: ein Trainingsschritt 4
Entscheidungen 4
Fehler im ersten Entwurf 5
Tests 6
Selbst ausprobieren 6

Aufbau

Die Struktur folgt dem Vorschlag der Angabe: Ein **Netz** besteht aus **Layern**, ein Layer aus **Neuronen**. Python und C# haben dieselben Klassen und Methoden, nur jeweils in der Schreibweise der Sprache (`update_weights` bzw. `UpdateWeights`).

```
classDiagram
    class NeuralNetwork {
        layers : list~Layer~
        learning_rate : float
        predict(inputs) outputs
        train(inputs, targets) cost
        fit(inputs, targets, epochs) costs
        dump_json(path)
        load(path)
        dump_txt(path)
        load_txt(path)
        dump_initfile(path, prefix)
    }
    class Layer {
        neurons : list~Neuron~
        forward(inputs) outputs
        backward(errors, learning_rate) previous_errors
    }
    class Neuron {
        weights : array
        bias : float
        inputs : array
```

```
        output : float
        activate(inputs) output
        activate_derivative() float
        update_weights(delta, learning_rate)
        update_bias(delta, learning_rate)
    }
    NeuralNetwork "1" *-- "1..*" Layer
    Layer "1" *-- "1..*" Neuron
```

Klasse	Aufgabe
`Neuron`	speichert Gewichte und Bias, berechnet $a = \sigma(w \cdot x + b)$ und merkt sich Input und Output für die Backpropagation
`Layer`	ruft alle Neuronen auf; `backward` berechnet die Deltas, aktualisiert die Neuronen und gibt den Fehler an den vorherigen Layer zurück
`NeuralNetwork`	verkettet die Layer, trainiert (`train` = ein Beispiel, `fit` = ganze Epochen) und speichert bzw. lädt Netze

Erzeugt wird ein Netz mit einer Liste von Größen, deren erster Eintrag die Anzahl der Inputs ist:

```python
# 2 inputs, 2 hidden neurons, 1 output; the seed makes the run reproducible
network = NeuralNetwork([2, 2, 1], learning_rate=0.1, seed=0)
network.fit(XOR_INPUTS, XOR_TARGETS, epochs=20000)
network.predict([1, 0])
```

In C# sieht derselbe Ablauf so aus:

```csharp
// 2 inputs, 2 hidden neurons, 1 output; the seed makes the run reproducible
var network = new NeuralNetwork([2, 2, 1], learningRate: 0.1, seed: 0);
network.Fit(Datasets.XorInputs, Datasets.XorTargets, epochs: 20000);
network.Predict([1, 0]);
```

Dateien

Aufgabe	Python (`python/neural_network/`)	C# (`csharp/`)
Aktivierungsfunktion	`activation.py`	`Activations.cs`
Neuron	`neuron.py`	`Neuron.cs`, Skalarprodukt in `NeuralNetworkMath.cs`
Layer	`layer.py`	`Layer.cs`

Aufgabe	Python (`python/neural_network/`)	C# (`csharp/`)
Netz	`network.py`	`NeuralNetwork.cs`
Dateiformate	`persistence.py`	`NetworkSerializer.cs`, `NetworkData.cs`
Trainingsdaten	`datasets.py`	`Datasets.cs`
Kommandozeile	`__main__.py`	`Program.cs`
Tests	`tests/` mit pytest	`SelfTest.cs`
Visualisierung	`visualization/`	—

Wichtige Codeteile

Neuron: Forward und Update

```python
def activate(self, inputs: FloatArray) -> float:
    self.inputs = inputs
    z = float(np.dot(self.weights, inputs)) + self.bias
    self.output = self._activation_fn(z)
    return self.output

def update_weights(self, delta: float, learning_rate: float) -> None:
    # dJ/dw_i = dJ/dz * dz/dw_i = delta * x_i
    self.weights -= learning_rate * delta * self.inputs
```

Layer: Backpropagation

Das ist die Umsetzung von Delta und Gradient für einen Layer:

```python
def backward(self, errors: FloatArray, learning_rate: float) -> FloatArray:
    previous_errors = np.zeros(self.input_size)

    for neuron, error in zip(self.neurons, errors, strict=True):
        # dJ/dz = dJ/da * da/dz
        delta = error * neuron.activate_derivative()

        # dz/dx_i = w_i; must use the weights from the forward pass,
        # so this happens before the update below
        previous_errors += delta * neuron.weights

        neuron.update_weights(delta, learning_rate)
        neuron.update_bias(delta, learning_rate)

    return previous_errors
```

Die C#-Version rechnet mit einfachen Schleifen statt mit numpy, der Ablauf ist derselbe:

```
public double[] Backward (double[] errors, double learningRate)
{
    double[] previousErrors = new double[InputSize];

    for (int n = 0; n < Neurons.Length; n++)
    {
        Neuron neuron = Neurons[n];

        // dJ/dz = dJ/da * da/dz
        double delta = errors[n] * neuron.ActivateDerivative();

        // dz/dx_i = w_i; must use the weights from the forward pass,
        // so this happens before the update below
        for (int i = 0; i < InputSize; i++)
        {
            previousErrors[i] += delta * neuron.Weights[i];
        }

        neuron.UpdateWeights(delta, learningRate);
        neuron.UpdateBias(delta, learningRate);
    }

    return previousErrors;
}
```

Netz: ein Trainingsschritt

```
def train(self, inputs: ArrayLike, targets: ArrayLike) -> float:
    expected = np.asarray(targets, dtype=np.float64)
    outputs = self.predict(inputs)

    # dJ/dy_hat for J = 1/2 * (y_hat - y)^2
    errors = outputs - expected
    for layer in reversed(self.layers):
        errors = layer.backward(errors, self.learning_rate)

    return 0.5 * float(np.sum((outputs - expected) ** 2))
```

Entscheidungen

- **Der Input ist kein Layer** — so steht es in der Angabe ($X = A^{[0]}$). Ein eigener Input-Layer mit Gewichten würde aus dem 2-2-1-Netz ein Netz mit drei Gewichtsschichten machen.
- **Neuron-Objekte statt Matrizen** — die Struktur der Angabe ist leichter nachzuvollziehen. Eine Gewichtsmatrix pro Layer wäre deutlich schneller: Ein 7-Segment-Training über 20 000 Epochen dauert in Python rund 18 Sekunden.
- **Startgewichte gleichverteilt in $[-1, 1)$ mit festem Seed** — Neuronen starten in unterschiedliche Richtungen, und jeder Lauf ist wiederholbar.

- **Numerisch stabile Sigmoid-Funktion** — für negative z wird $\frac{e^z}{1+e^z}$ gerechnet, damit `exp` nie mit einer großen positiven Zahl aufgerufen wird und überläuft.
- **Eine Kommandozeile für alles** — `xor`, `seven-segment` und `visualize` (Python) bzw. `selftest` und `load` (C#) statt einzelner Skripte.
- **C# ohne Zusatzpakete** — single-threaded und mit einfachen Schleifen, damit man jeden Rechenschritt im Debugger verfolgen kann. Statt eines Test-Frameworks (das ein NuGet-Paket bräuchte) prüft ein eingebauter Selbsttest dieselben Punkte wie die Python-Tests.
- **Skalarprodukt als einfache Schleife** — die SIMD-Variante aus dem ersten Entwurf (`System.Numerics.Vector`) wäre schneller, versteckt aber die Rechnung. Bei höchstens 16 Inputs pro Neuron reicht die Schleife: Das 7-Segment-Training dauert in C# rund eine Sekunde.
- **Byte-gleiche Dateien in beiden Sprachen** — C# schreibt Zahlen genau wie Pythons `repr`; .NET allein würde etwa `1E-05` statt `1e-05` schreiben. Ein Dateivergleich zeigt so sofort, ob beide Seiten dasselbe Netz speichern.
- **Eigener Zufallsgenerator je Sprache** — C# verwendet `System.Random`, Python den Generator von numpy. Gleiche Seeds ergeben deshalb andere Startgewichte; den Generator von numpy nachzubauen wäre viel Code ohne Nutzen für das Verständnis.

Fehler im ersten Entwurf

Der erste C#-Entwurf hatte die richtige Struktur, aber mehrere Fehler, die das Training verhindert hätten. Sie sind in beiden Versionen behoben:

Stelle	Fehler	Richtig
`Neuron.Activate`	Sigmoid fehlte, die Ausgabe war nur z	$a = \sigma(z)$
`ActivateDerivative`	Sigmoid wurde ein zweites Mal angewendet	$a(1-a)$ mit der gespeicherten Ausgabe
`UpdateWeights` / `UpdateBias`	`w += gradient * w`, Lernrate unbenutzt	$w \mathrel{-}= \alpha\,\delta\,x$ bzw. $b \mathrel{-}= \alpha\,\delta$
`NeuralNetwork`	Input-Layer mit eigenen Gewichten; der Hidden-Layer bekam die Input-Größe statt der Neuronenzahl davor	Input ist $A^{[0]}$, jeder Layer bekommt die Größe des vorherigen
`Layer`	Felder für die Größen wurden nie gesetzt	Größen aus den Neuronen ableiten
Startgewichte	nur positiv in $[0, 1)$	$[-1, 1)$
`Neuron.cs`	`using NumSharp` und `using System.Runtime.InteropServices.Swift`, beide unbenutzt; NumSharp wäre ein NuGet-Paket	entfernt
Python-Gerüst	`np.random_uniform` gibt es nicht, `for i in neuronSize` ohne `range`, Tippfehler `active` statt `activate`	`rng.uniform`, `range(...)`, `activate` mit Rückgabewert

Tests

Die Python-Version hat 27 automatische Tests (`pytest`), die C#-Version einen eingebauten Selbsttest mit 17 Prüfungen (`dotnet run -- selftest`):

Test	Prüft	Python	C#
Gradienten-Check	Backpropagation gegen numerische Ableitung, drei Netzgrößen	✓	✓
Forward-Pass	Ergebnis gegen eine von Hand nachgerechnete Formel	✓	✓
Konvergenz	XOR lernt alle 4 Fälle, der 7-Segment-Decoder alle 10 Ziffern	✓	✓
Export/Import	JSON und TXT neu laden ergibt bitgleiche Vorhersagen; C-Header enthält jeden Wert	✓	✓
Fehlerfälle	falsche Layer-Größen, falsche Input-Länge, kaputte Dateien werden abgelehnt	✓	✓
Zahlenformat	jede Zahl wird wie Pythons `repr` geschrieben und exakt zurückgelesen	—	✓
Austausch	Modelle der anderen Sprache laden und sagen richtig vorher; die daraus geschriebenen Dateien sind byte-gleich	✓	✓
Visualisierung	die erzeugte Seite enthält gültige Daten für alle Szenarien	✓	—

Selbst ausprobieren

Python im Ordner `python/` des Repos:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m neural_network xor
.venv\Scripts\python   -m   neural_network   seven-segment   --export   ..
\models\python
.venv\Scripts\python -m neural_network visualize
.venv\Scripts\python -m pytest
```

C# im Ordner `csharp/` (benötigt das .NET 10 SDK, keine weiteren Pakete):

```
dotnet run -- xor
dotnet run -- seven-segment --export ..\models\csharp
dotnet run -- load ..\models\python\xor.json
dotnet run -- selftest
```